

C2ISOLATE

Erkennung von C2 Verhalten im Netzwerk mit
ML-Algorithmen

Silas Burkhard

Inhaltsverzeichnis

Einleitung	2
Änderungstabelle.....	2
Projektbeschreibung	2
Risiken	2
Hauptteil	2
C2 Kommunikationstechniken	2
ML-Algorithmen	3
Isolation Forest	4
Random Forest.....	4
K-means Clustering	4
MLP	5
Featureliste	6
Tagesreflexionen	11
Tag 1, 03.06	11
Tag 2, 04.06	11
Tag 3, 05.06	11
Tag 4, 10.06	11
Tag 5, 11.06	11
Tag 6, 12.06	11
Tag 7, 17.06	11
Tag 8, 18.06	11
Tag 9, 19.06	11
Tag 10, 24.06	11
Tag 11, 25.06	11
Tag 12, 26.06	11

Einleitung

Änderungstabelle

Datum	ÄNDERUNGSBESCHREIB	ORT
03.06.2026	Dokument erstellt	
04.06.2026	Grundlegende Struktur	Einleitung, Hauptteil, Tagesreflexionen
05.06.2026	Kommunikationstechniken und Featureliste	Hauptteil

Projektbeschreibung

In meinem Projekt will ich C2 Verhalten im Netzwerkverkehr mithilfe von ML-Algorithmen erkennen. Dazu muss ich herausfinden, was für Techniken angreifer nutzen, um ihren Datenverkehr zu verschleiern, was ich für einen ML-Algorithmus nutzen werde und welche Features des Datenverkehrs ich brauche.

Ich kam auf dieses Thema, da ich mir zuerst überlegt hatte, allgemein den Netzwerkverkehr mit Machine Learning zu überwachen, aber dann einfach Port-Scans oder Syn-Floods zu erkennen fand ich langweilig. Deshalb habe ich mich dafür entschieden etwas konkreteres zu machen, und C2 Verhalten zu erkennen wäre sicher eine Challenge, da dieser Verkehr dazu gedacht ist unerkannt zu bleiben.

Risiken

Es gibt einige Risiken in meinem Projekt. Zum Beispiel kann es sein, dass ich gar nicht einen geeigneten Algorithmus finde, der mir weiterhilft. Oder es könnte sein, dass der Verkehr den ich als C2 simuliere gar nicht wirklich mit echtem C2 Verkehr vergleichbar ist. Ich weiss auch nicht, welche Features ich brauche, um zuverlässige Resultate zu erzielen. Auch Probleme beim Coden können viel Zeit kosten.

Hauptteil

C2 Kommunikationstechniken

Um später verdächtigen Datenverkehr zu erkennen, muss ich wissen wie eine C2-Struktur kommuniziert. Eine einfache C2-Architektur sieht meistens ungefähr so aus:

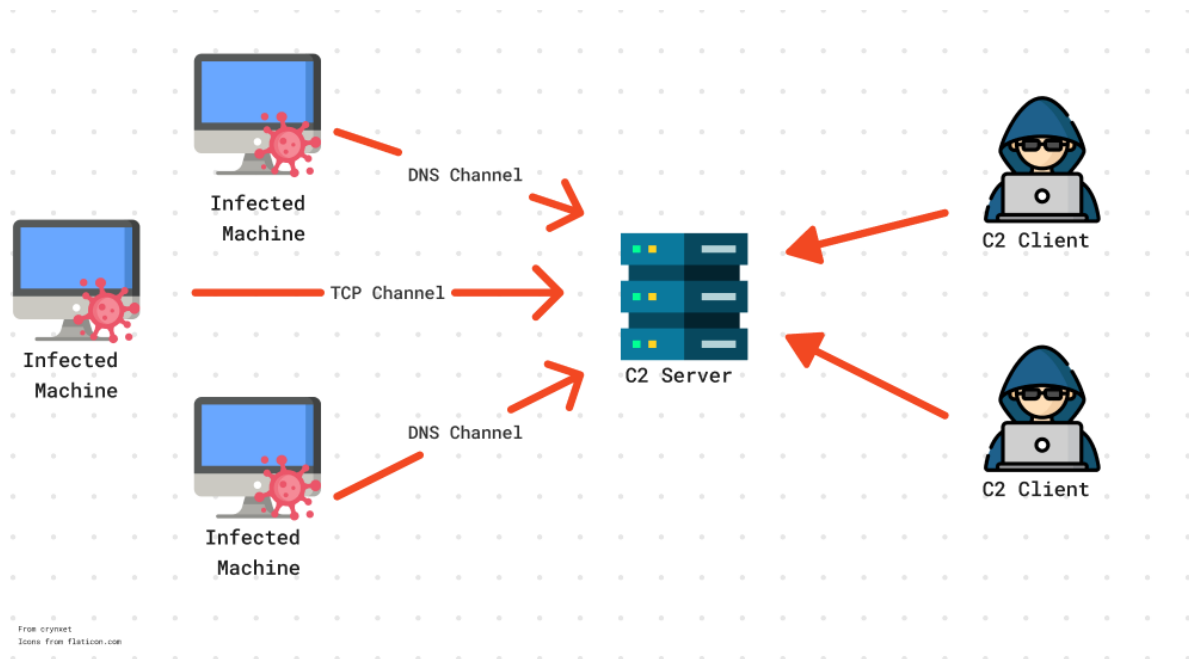


Abbildung 1: C2 Struktur [Quelle](#)

Dabei stehen die infizierten Maschinen direkt im Kontakt mit dem C2 Server. Angreifer können dann über den C2 Server Befehle ausführen oder Angriffe koordinieren. Für die Kommunikation zwischen der infizierten Maschine und dem C2 Server gibt es viele Möglichkeiten. Einige davon sind:

- DNS-Tunneling
- Socket Filter
- Protocol/Service Impersonation

Mit diesen Techniken kann der Datenfluss zwischen dem C2 Server und dem Opfer System versteckt werden. Die meist genutzten Protokolle zur C2 Kommunikation sind HTTP/HTTPS, DNS, IRC, FTP und E-Mail Protokolle.

Eine verbreitete Methode um den Opfersystemen Anweisungen zu schicken, ist Beaconsing. Dabei schickt das Opfer regelmässige Anfragen an den C2 Server um Anweisungen zu bekommen. Das primäre Ziel von diesem Projekt ist es, genau diesen Datenverkehr zu erkennen. Angreifer nutzen Taktiken wie Jitter, wo ein zufälliger Wert der normalen Wartezeit hinzugefügt wird, damit der Datenverkehr weniger Roboterhaft wirkt.

ML-Algorithmen

Ich habe mich über einige ML-Algorithmen informiert, die sich für mein Projekt eignen könnten. Es gibt noch viel mehr und auch viel spezialisiertere Algorithmen als diese, aber ich habe mich über diese informiert, weil ich schon darüber gehört oder schon damit gearbeitet habe.

Isolation Forest

Isolation Forest ist ein Algorithmus zur Anomalie-Erkennung. Um Anomalien zu finden, wählt der Algorithmus ein Feature von dem Datensatz ausgewählt und dann an einer zufälligen Stelle halbiert. Dieser Prozess der Halbierung wird wiederholt, bis eine von zwei Bedingungen erfüllt sind: Entweder ist an jedem Punkt nur noch ein Datentupel oder es wurde eine vorbestimmte Tiefe erreicht. Jetzt hat man einen Baum von Daten, und an jedem Ende eines Astes hat es nur noch wenige bis ein Datentupel. Anomalien brauchen weniger Spaltungen als die anderen Daten um sich vollständig abzugrenzen, dadurch findet man sie. Ein solcher Baum wird auch wiederholt erstellt, um die Sicherheit des Resultates zu verbessern.

Dieser Algorithmus ist auch gut mit grossen Datenmengen und wird auch in der Cybersecurity verwendet. Ausserdem braucht der Algorithmus kein Training, es ist ein „unsupervised Learning Algorithm“.

Random Forest

Random Forest ist ein Algorithmus, der sich wie Isolation Forest auch Decision Trees zunutze macht. Dabei ist Random Forest nicht wie Isolation Forest auf Anomalien ausgelegt, sondern auf Klassifizierung und Regression. Random Forest ist auch ein supervised Learning Algorithm, es braucht also gekennzeichnete Trainingsdaten.

K-means Clustering

K-means Clustering ist ein Algorithmus zur Gruppierung von Daten. Dabei wird angenommen, dass die Gruppen näherungsweise Spherenförmig sind und man muss die Anzahl von Gruppen wissen. K-means Clustering ist wie der Isolation Forest ein unsupervised Learning Algorithm, und braucht kein Training mit gekennzeichneten Übungsdaten.

Bei K-means Clustering werden zunächst zufällige Startpunkte als Zentrum für die Gruppen ausgewählt. Danach versucht der Algorithmus die Zentren so zu verbessern, das mehr Punkte einen kleineren Abstand zu ihrem Zentrum haben. Dies wird mehrmals wiederholt, bis man ein stabiles Resultat hat.

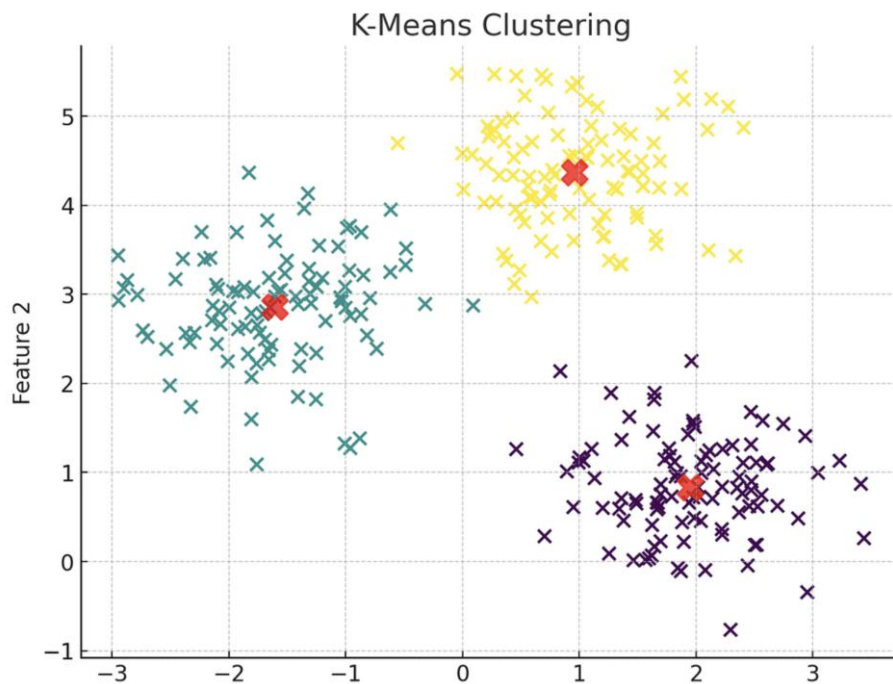


Abbildung 2: K-means Clustering [Quelle](#)

Zuerst dachte ich, dass ich diesen Algorithmus wahrscheinlich noch gut nutzen könnte, jedoch sind meine Daten fast nie zuverlässig Spherenförmig aufgeteilt. Deshalb werde ich diesen Algorithmus eher nicht nutzen.

MLP

MLP steht für Multilayer Perceptron, und ist ein künstliches Neuronales Netzwerk, dem durch Lernen Zusammenhänge zwischen Daten, klassifizierung, Anomalieerkennung und weiteres beigebracht werden kann. Durch das ist es ein sehr ein starkes Tool, wenn man die Trainingsdaten und Rechenleistung dafür hat.

Ein MLP hat immer den gleichen Aufbau: Zuerst eine Schicht von Input-Neuronen, die Anzahl von diesen korrespondiert direkt mit der Anzahl an Dimensionen der Daten, die der MLP entgegennehmen kann. Dann kommt eine definierte Anzahl an verborgenen Schichten, bei diesen werden schlussendlich die Kalkulationen gemacht. Am Ende ist noch eine letzte Schicht von Neuronen, der Output-Layer. Die Struktur kann zum Beispiel wie im Bild unten aussehen:

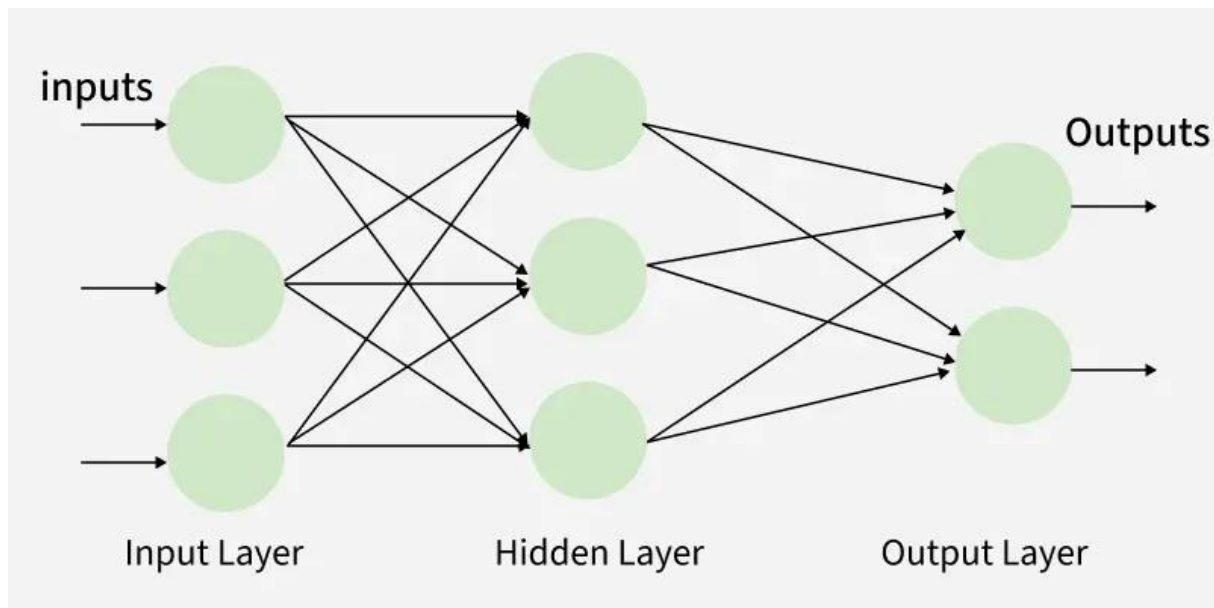


Abbildung 3: Struktur eines MLP [Quelle](#)

Ich habe mich eher gegen den Multilayer Perceptron entschieden, da ich dann grosse Mengen von Übungsdaten haben müsste, um ein Model zu trainieren. Es ist aber sicher eine interessante Möglichkeit das Problem anzugehen, wenn man genügend Zeit und Informationen hat.

Featureliste

Da ich mich für die Python Library `nfstream` entschieden habe, bekomme ich nun pro Flow 86 Features über diesen Flow. Vieles davon wird nicht aufzeigen, ob ein Flow eine Kommunikation mit einem C2 Server ist. Deshalb muss ich ein Verfahren haben, um für jedes Feature zu prüfen, ob es mir hilft, bösartiger Verkehr zu erkennen.

Aus diesem Grund habe ich mich dafür entschieden eine Dimensionsreduzierende Funktion zu nutzen, wodurch man die Daten auf einen Zwei-Dimensionalen Graph visualisieren kann. Zuerst wollte ich t-SNE nutzen, da ich über dieses Verfahren schon in meiner Recherche über die verschiedenen ML-Algorithmen gestossen bin. Aber nachdem ich nochmals ein wenig darüber nachgeforscht habe, habe ich herausgefunden, dass t-SNE nicht sehr gut ist für sehr grosse Datensätze. Deshalb bin ich auf PCA, Principal Component Analysis, umgestiegen. PCA ist linear, nicht so wie t-SNE. Ich implementierte PCA in meinem Script, sodass ich am Ende einen Graphen zeichnen konnte, auf dem alle Flows sichtbar sind. Die Flows die von meinem C2 Traffic Simulator waren, habe ich rot gezeichnet, den Rest grau. Nun kann ich Schritt für Schritt Features entfernen und den Graph vergleichen. Wenn sich die roten Punkte besser gruppieren und sich von den grauen Punkten abgrenzen, ist das Feature hinderlich zum C2 Traffic zu erkennen, und ich werde es permanent entfernen. Wenn sich am Graph nicht wirklich etwas verändert, ist das Feature auch irrelevant und wird entfernt. Wenn aber sich die Gruppierung der roten Punkte verschlechtert und sie besser einblenden ist das Feature relevant, und ich werde es nicht entfernen.

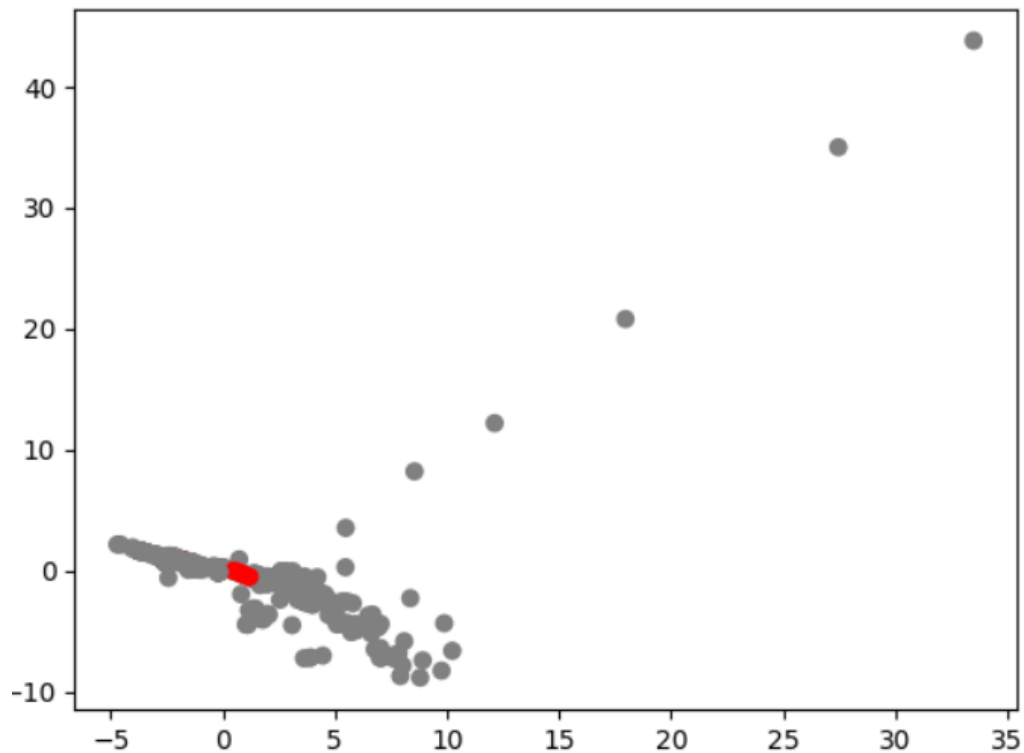


Abbildung 4: Ursprungsgraph mit allen Features

Dieser erste Graph wurde mit allen Features erstellt. Man sieht, dass sich die roten Punkte gut mit den grauen vermischen, was nicht gut ist.

Ein paar Features streiche ich schon ohne sie zu überprüfen, da ich weiss, dass sie nicht relevant sind. Folgende Features habe ich gestrichen:

- id
- expiration_id
- ip_version
- vlan_id
- tunnel_id
- application_is_guessed
- application_confidence

Ohne diese Features sieht der Graph so aus:

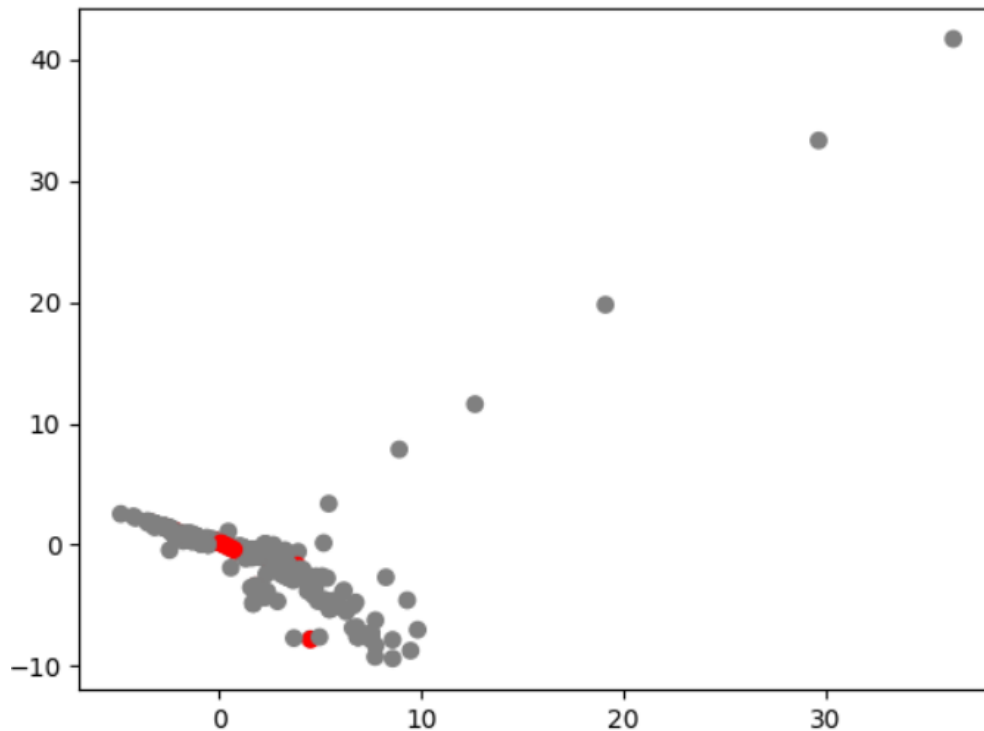


Abbildung 5: Graph ohne unwichtige Features

Im Vergleich zum ersten Graph hat sich nicht sehr viel verändert. Dadurch merkt man wie oben beschrieben, dass diese Features wie ich angenommen habe, unwichtig sind.

Nachdem ich diesen Prozess ein paar mal wiederholt habe, ist mir aufgefallen, dass meine gekennzeichneten roten Punkte gar nicht stimmen können. Ich nutzte ChatGPT um den Fehler in meinem Code zu finden, ich hatte verschiedene Arten von Indexen genutzt. Einerseits der Index von `nfstream`, der nicht unbedingt mit dem positionalen Index von Arrays übereinstimmt. Andererseits auch noch der Index vom Pandas Objekt `DataFrame`, der auch nicht mit den anderen zwei Indexen übereinstimmen muss. Nachdem ich dieses Problem gelöst hatte und weitere Features überprüft habe, fiel mir auf, dass jedes Feature das ich entfernte oder hinzufügte fast keinen Einfluss darauf hatte, wie gut die roten Punkte positioniert waren.

Deshalb entschied ich mich, es noch mit dem t-SNE Algorithmus zu versuchen, da die Linearität von PCA ein Problem sein könnte. Beim Graph unten sieht man, dass die Punkte schon viel besser gruppiert waren.

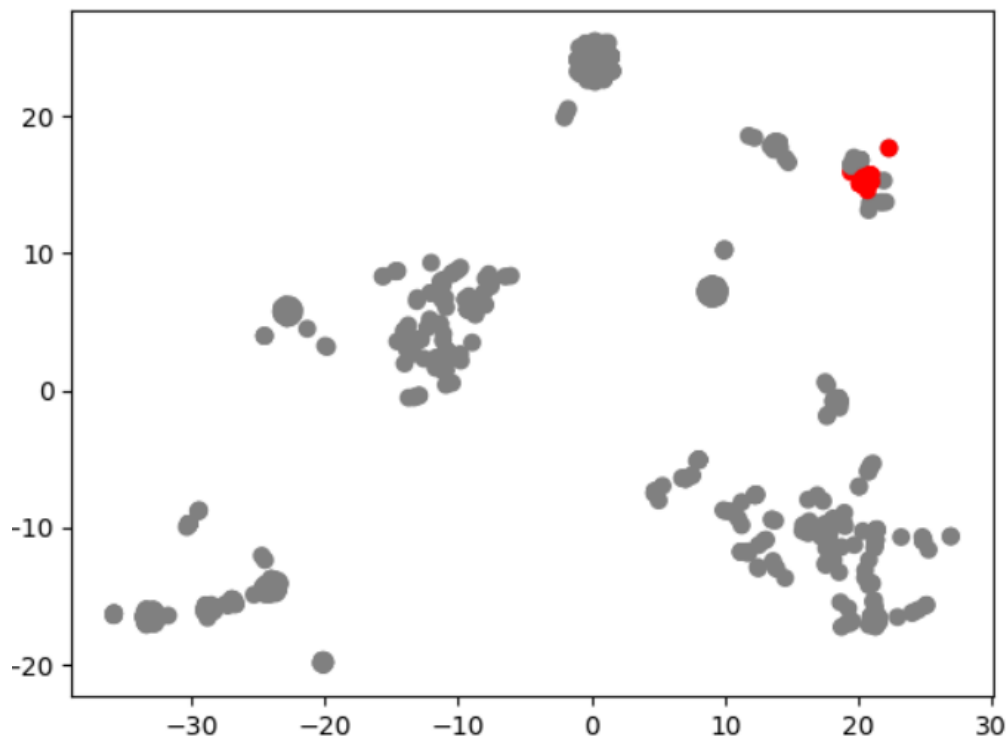


Abbildung 6: t-SNE mit fast allen Features

Nachdem ich viele verschiedene Varianten ausprobiert habe, habe ich mich für die folgenden Features entschieden:

- `bidirectional_duration_ms`: Dauer des Flows von beiden Seiten in Millisekunden
- `bidirectional_packets`: Anzahl an Paketen im Flow
- `bidirectional_bytes`: Anzahl an Bytes im Flow
- `src2dst_bytes`: Anzahl an src2dst Bytes im Flow
- `dst2src_bytes`: Anzahl an dst2src Bytes im Flow
- `dst2src_max_ps`: Maximale dst2src Paketgröße
- `src2dst_max_ps`: Maximale src2dst Paketgröße
- `src2dst_max_piat_ms`: Maximale Zeit zwischen src2dst Paketen
- `dst2src_max_piat_ms`: Maximale Zeit zwischen dst2src Paketen

Ich habe Features gruppenweise auf ihren Einfluss überprüft. Zum Beispiel alle Features die nur die Anzahl einer bestimmten Flag zählen oder alle Features die die Startzeit des Flows beinhalten. Graphen mit den ausgewählten Features sehen so aus:

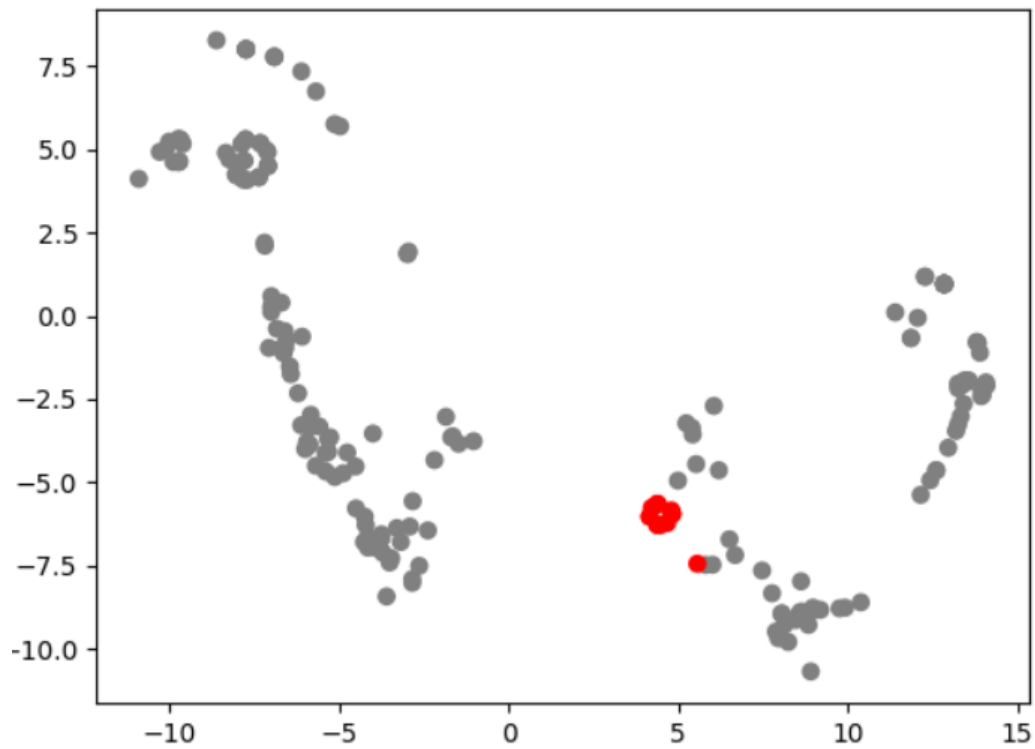


Abbildung 7: Features mit 5k Paketen

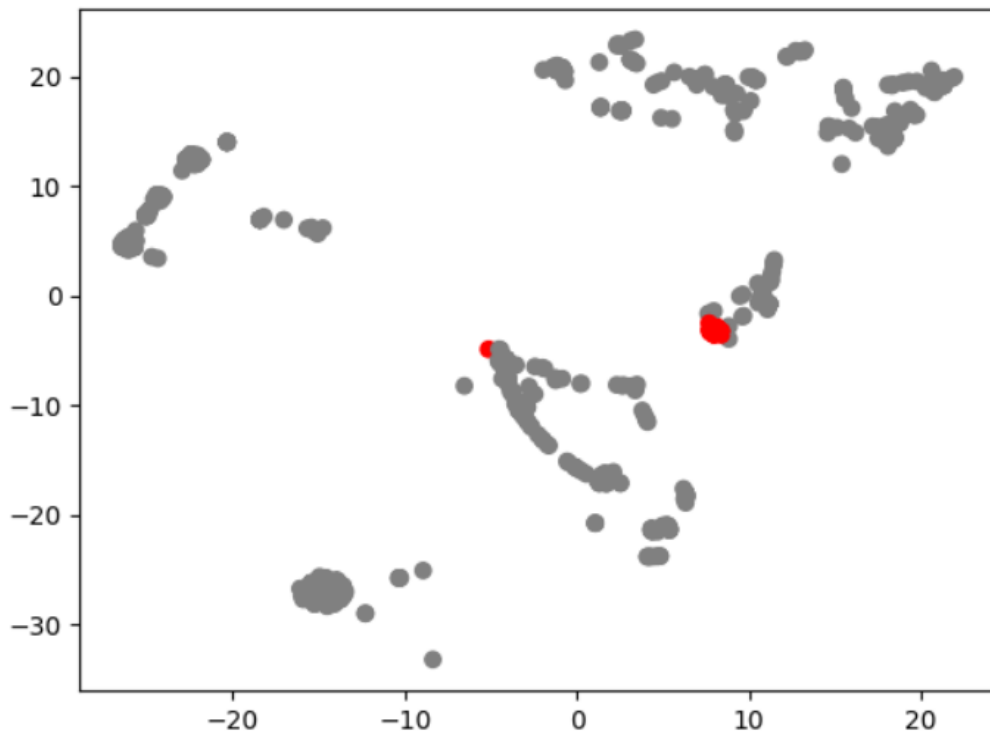


Abbildung 8: Features mit 400k Paketen

Man sieht, dass die roten Punkte sich gruppieren, es ist aber auch keinesfalls perfekt. Ich werde später im Projekt die Liste der ausgewählten Features auch noch verändern.

Prototyp

Tagesreflexionen

Tag 1, 03.06

Am ersten Tag der Projektarbeit habe ich mein Github Repository und Project erstellt. Dann musste ich meine Meilensteine aufschreiben und ich habe mir noch ein optionales Ziel überlegt. Danach habe ich angefangen mit der Planung und danach meine Issues hinzugefügt.

Tag 2, 04.06

Am zweiten Tag habe ich einige Issues zur Informationsbeschaffung abgearbeitet. Ich habe angeschaut was für Techniken und Taktiken von Angreifern verwendet werden, um nicht erkannt zu werden. Ich habe auch nach öffentlichen Tools gesucht, die etwas ähnliches bewerkstelligen, um ein wenig Inspiration zu kriegen. Als Letztes habe ich nach Informationen über verschiedene ML-Algorithmen gesucht, um zu entscheiden welcher der beste für mein Projekt ist.

Tag 3, 05.06

Am dritten Tag habe ich die letzten Issues zur Informationsbeschaffung bearbeitet. Ich verwendete den Grossteil des Tages darauf, herauszufinden welche Features nötig oder unnötig sind. Ich hatte einige Probleme mit meinen Scripts, da ich auch nicht so erfahren darin bin, mit grossen Datenmengen und spezifischen Librarys zu arbeiten.

Tag 4, 10.06

Tag 5, 11.06

Tag 6, 12.06

Tag 7, 17.06

Tag 8, 18.06

Tag 9, 19.06

Tag 10, 24.06

Tag 11, 25.06

Tag 12, 26.06